

Lab 6

Nicholas Storti

July 29, 2026

1 Reduction

1.1 Output

```
Setting up the problem...0.000006 s
Input size = 512
Allocating device variables...0.159924 s
Copying data from host to device...0.000032 s
Launching kernel...0.010355 s
Copying data from device to host...0.000019 s
Verifying results...TEST PASSED
```

```
Setting up the problem...0.000012 s
Input size = 1024
Allocating device variables...0.137162 s
Copying data from host to device...0.000032 s
Launching kernel...0.010324 s
Copying data from device to host...0.000019 s
Verifying results...TEST PASSED
```

```
Setting up the problem...0.000041 s
Input size = 5000
Allocating device variables...0.137643 s
Copying data from host to device...0.000034 s
Launching kernel...0.010342 s
Copying data from device to host...0.000021 s
Verifying results...TEST PASSED
```

```
Setting up the problem...0.000749 s
Input size = 100000
Allocating device variables...0.134589 s
Copying data from host to device...0.000070 s
Launching kernel...0.010366 s
Copying data from device to host...0.000013 s
Verifying results...TEST PASSED
```

```
Setting up the problem...0.003736 s
Input size = 500000
Allocating device variables...0.135644 s
```

```

Copying data from host to device...0.000169 s
Launching kernel...0.010399 s
Copying data from device to host...0.000014 s
Verifying results...TEST PASSED

```

```

Setting up the problem...0.006859 s
Input size = 1000000
Allocating device variables...0.136536 s
Copying data from host to device...0.000254 s
Launching kernel...0.010344 s
Copying data from device to host...0.000014 s
Verifying results...TEST PASSED

```

```

Setting up the problem...0.013095 s
Input size = 2000000
Allocating device variables...0.134931 s
Copying data from host to device...0.000425 s
Launching kernel...0.010344 s
Copying data from device to host...0.000019 s
Verifying results...TEST PASSED

```

1.2 Answer Section

1. How many times does a single thread block synchronize to reduce its portion of the array to a single value?

There are 512 threads in a thread block. On each iteration of the for-loop for the reduction tree, each of the 512 threads synchronizes. If the number of elements to process reduces by half on each iteration of the for-loop, then $\log_2(512)$ for-loop iterations are performed. A single thread block then performs $(512 \frac{\text{synchronizations}}{\text{iteration}})(\log_2(512) \text{ iterations}) = (512)(9) = 4608$ synchronizations to reduce its portion of the array to a single value.

2. What is the minimum, maximum, and average number of "real" operations that a thread will perform? "Real" operations are those that directly contribute to the final reduction value.

For my implementation of the reduction kernel, each thread loads two elements from global memory, adds them together to complete the first level of the reduction tree, then stores the result in a shared memory array that has the same size as the number of threads in the thread block. When the for-loop starts, only half of the threads compute the next level of the reduction tree. Since each thread performs at least one addition to compute the first level of the reduction tree, the minimum number of "real" operations that a thread will perform is 1. Each block processes a 1024 element section of the input array. There will be one thread in the block that is active on every iteration of the for-loop. The maximum number of "real" operations that this thread will perform is $\log_2(1024) = 10$ operations. The average number of "real" operations is $\frac{1+10}{2} = 5.5$ operations.

2 Prefix Scan

2.1 Output

```

Setting up the problem...0.000013 s
Input size = 1000
Allocating device variables...0.166151 s
Copying data from host to device...0.000034 s
Launching kernel...0.016869 s

```

Copying data from device to host...0.000014 s
Verifying results...TEST PASSED

Setting up the problem...0.000022 s
Input size = 2000
Allocating device variables...0.134964 s
Copying data from host to device...0.000042 s
Launching kernel...0.016448 s
Copying data from device to host...0.000016 s
Verifying results...TEST PASSED

Setting up the problem...0.000761 s
Input size = 100000
Allocating device variables...0.141793 s
Copying data from host to device...0.000071 s
Launching kernel...0.016484 s
Copying data from device to host...0.000191 s
Verifying results...TEST PASSED

Setting up the problem...0.003743 s
Input size = 500000
Allocating device variables...0.137189 s
Copying data from host to device...0.000167 s
Launching kernel...0.016513 s
Copying data from device to host...0.000889 s
Verifying results...TEST PASSED

Setting up the problem...0.006852 s
Input size = 1000000
Allocating device variables...0.135980 s
Copying data from host to device...0.000261 s
Launching kernel...0.016640 s
Copying data from device to host...0.001061 s
Verifying results...TEST PASSED

Setting up the problem...0.031773 s
Input size = 5000000
Allocating device variables...0.142516 s
Copying data from host to device...0.000945 s
Launching kernel...0.016712 s
Copying data from device to host...0.002563 s
Verifying results...TEST PASSED

Setting up the problem...0.062833 s
Input size = 10000000
Allocating device variables...0.143745 s
Copying data from host to device...0.001768 s
Launching kernel...0.016744 s

```
Copying data from device to host...0.005084 s
Verifying results...TEST PASSED
```

```
Setting up the problem...0.315241 s
Input size = 50000000
Allocating device variables...0.143893 s
Copying data from host to device...0.008548 s
Launching kernel...0.018036 s
Copying data from device to host...0.022835 s
Verifying results...TEST PASSED
```

```
Setting up the problem...0.628785 s
Input size = 100000000
Allocating device variables...0.139372 s
Copying data from host to device...0.016965 s
Launching kernel...0.019665 s
Copying data from device to host...0.043368 s
Verifying results...TEST PASSED
```

```
Setting up the problem...3.140867 s
Input size = 500000000
Allocating device variables...0.158835 s
Copying data from host to device...0.084609 s
Launching kernel...0.029150 s
Copying data from device to host...0.232920 s
Verifying results...TEST PASSED
```

2.2 Answer Section

1. Describe how you handled arrays not a power of two in size and all performance-enhancing optimizations you added.

For arrays not a power of two in size, there are more threads than input elements. For the extra threads, those threads wrote 0's to the extra shared memory locations. In terms of performance enhancing optimizations, to decrease memory access latency, I used shared memory to store the segment of the input array being processed by each block. I also used the three-phase Brent-Kung scan algorithm to increase work-efficiency. In this algorithm, each thread processes a small amount of the input segment serially first. This allows for more of the threads to be utilized. In the second phase, the Brent-Kung scan algorithm is performed on the last element from each of the serial segments. In the third phase, offsets are then added to each element to complete the scan.

I also made a small modification to the verify function in support.cu to allow for testing of larger input arrays than 1,000,000. I changed the data type of the sum variable from float to double. This allowed me to test input arrays up to 500,000,000 elements.